

CSCI 5709: Advanced Web Services

Assignment 2

Project: CampusCart - A Campus Buy-and-Sell Marketplace

Student: Mahek Dilipbhai Shah

Banner ID: B01043202

Group: Group 8

Date: June 27, 2026

0. Introduction and Scope

CampusCart is a campus-only buy-and-sell marketplace for university students, as defined in the Group 8 project proposal. This report designs and documents the RESTful API for three of CampusCart's core features, defines the security-related endpoints needed to protect them, sketches the underlying data models, and describes a working implementation of two endpoints from one of the three features.

The three core features selected for this assignment are:

- Core Feature 2 - Product Listing Management (create, edit, delete, view listings)
- Core Feature 3 - Browse, Search & Filter (public discovery of listings)
- Core Feature 4 - Favorite / Saved Listings (bookmarking listings for later)

As instructed, user registration and login are documented separately under Section 2 (Security-Related Endpoints) rather than counted as core features.

All endpoints below are prefixed with the API base path /api and assume JSON request/response bodies (Content-Type: application/json) unless otherwise noted.

1. Core Feature API Endpoints

1.1 Core Feature 2 - Product Listing Management

Authenticated students can create, view, edit, and delete their own product listings. Each listing stores a title, description, price, category, condition, optional images, and a reference to the seller. This maps directly to the proposal's Core Feature 2.

POST /api/listings [REQUIRED ENDPOINT - IMPLEMENTED & DEPLOYED]

Create a new product listing. Only authenticated users with role 'student' or 'admin' may create listings.

Authentication: Required - Bearer JWT in Authorization header **Authorization:** Any authenticated student (role-based, not ownership-based, since this creates a new resource)

Request Parameters / Body:

Field	Type	Required?	Notes
title	String	Yes	1-120 characters
description	String	Yes	1-2000 characters
price	Number	Yes	>= 0
category	String (enum)	Yes	Textbooks Electronics Furniture Accessories Other
condition	String (enum)	Yes	New Like New Good Fair
images	Array<String>	No	Up to 6 image URLs

Success Response: 201 Created

```
{
  "listing": {
    "_id": "665f1...",
    "title": "Calculus: Early Transcendentals 8th Ed.",
    "description": "Gently used, like new condition.",
    "price": 45,
    "category": "Textbooks",
    "condition": "Like New",
    "images": [],
    "seller": "665f0...",
    "status": "active",
    "views": 0,
    "createdAt": "2026-06-27T19:34:42.232Z"
  }
}
```

Error Responses:

Status Code	Condition	Error Message (JSON)
-------------	-----------	----------------------

400 Bad Request	Missing/invalid field (e.g. blank title, negative price, invalid category)	{"error":"ValidationError","message":"One or more fields are invalid","details":[{"field":"price","message":"price must be a number >= 0"}]}
401 Unauthorized	Missing or invalid JWT	{"error":"Unauthorized","message":"Missing or malformed Authorization header. Expected: Bearer <token>"}
403 Forbidden	Account suspended by an admin	{"error":"Forbidden","message":"This account has been suspended"}

GET /api/listings/:id

Retrieve a single listing by id, including seller's public profile fields. Increments the listing's view counter (supports Seller Dashboard view metrics).

Authentication: Not required (public) **Authorization:** N/A - read-only public resource

Request Parameters / Body:

Field	Type	Required?	Notes
id	ObjectId (path param)	Yes	Must be a valid 24-character Mongo ObjectId

Success Response: 200 OK

```
{
  "listing": {
    "_id": "665f1...",
    "title": "Calculus: Early Transcendentals 8th Ed.",
    "price": 45,
    "views": 12,
    "seller": { "fullName": "Jane Smith", "universityEmail": "j.smith@dal.ca" }
  }
}
```

Error Responses:

Status Code	Condition	Error Message (JSON)
400 Bad Request	id is not a valid ObjectId	{"error":"ValidationError","message":"One or more fields are invalid","details":[{"field":"id","message":"Invalid listing id"}]}
404 Not Found	No listing exists with that id	{"error":"NotFound","message":"Listing not found"}

PUT /api/listings/:id

Edit an existing listing. Only the original seller (or an admin) may edit it. Demonstrates ownership-based access control (broken-access-control mitigation).

Authentication: Required - Bearer JWT **Authorization:** Owner of the listing OR admin role

Request Parameters / Body:

Field	Type	Required?	Notes
title	String	No	1-120 characters
description	String	No	1-2000 characters
price	Number	No	>= 0
category	String (enum)	No	See Core Feature 2 categories
condition	String (enum)	No	New Like New Good Fair
status	String (enum)	No	active sold flagged removed

Success Response: 200 OK

```
{
  "listing": { "_id": "665f1...", "title": "Updated title", "price": 40, "status": "sold" }
}
```

Error Responses:

Status Code	Condition	Error Message (JSON)
401 Unauthorized	Missing/invalid JWT	{"error":"Unauthorized","message":"Invalid JWT"}
403 Forbidden	Authenticated user does not own this listing	{"error":"Forbidden","message":"You can only edit your own listings"}
404 Not Found	Listing id does not exist	{"error":"NotFound","message":"Listing not found"}

DELETE /api/listings/:id

Permanently delete a listing. Only the original seller (or an admin) may delete it.

Authentication: Required - Bearer JWT **Authorization:** Owner of the listing OR admin role

Request Parameters / Body:

Field	Type	Required?	Notes
id	ObjectId (path param)	Yes	

Success Response: 200 OK

```
{ "message": "Listing deleted successfully" }
```

Error Responses:

Status Code	Condition	Error Message (JSON)
401 Unauthorized	Missing/invalid JWT	{"error":"Unauthorized","message":"Invalid JWT"}
403 Forbidden	Authenticated user does not own this listing	{"error":"Forbidden","message":"You can only delete your own listings"}
404 Not Found	Listing id does not exist	{"error":"NotFound","message":"Listing not found"}

1.2 Core Feature 3 - Browse, Search & Filter

All visitors, including unauthenticated ones, can browse listings on the main marketplace page, search by keyword, filter by category/condition/price, sort, and page through results.

GET /api/listings [REQUIRED ENDPOINT - IMPLEMENTED & DEPLOYED]

Browse, search, filter, sort, and paginate active listings. This is the primary endpoint behind the marketplace home page.

Authentication: Not required (public) **Authorization:** N/A - read-only public resource; results are always limited to status='active' listings

Request Parameters / Body:

Field	Type	Required?	Notes
q	String (query)	No	Free-text search across title & description (MongoDB text index)
category	String (query)	No	Textbooks Electronics Furniture Accessories Other
condition	String (query)	No	New Like New Good Fair
minPrice / maxPrice	Number (query)	No	>= 0
sort	String (query)	No	newest (default) oldest price_asc price_desc
page	Integer (query)	No	Default 1
limit	Integer (query)	No	Default 12, max 50

Success Response: 200 OK

```
{
  "listings": [ { "_id": "665f1...", "title": "Calculus...", "price": 45, "category": "Textbooks" } ],
  "pagination": { "page": 1, "limit": 12, "total": 6, "totalPages": 1 }
}
```

Error Responses:

Status Code	Condition	Error Message (JSON)
400 Bad Request	Invalid category/condition value, or non-numeric page/limit/minPrice/maxPrice	{"error": "ValidationError", "message": "category must be one of: Textbooks, Electronics, Furniture, Accessories, Other"}

GET /api/listings/categories

Returns the static list of valid categories and conditions so the front end can render filter dropdowns without hardcoding enum values.

Authentication: Not required (public) **Authorization:** N/A

Success Response: 200 OK

```
{
  "categories": ["Textbooks", "Electronics", "Furniture", "Accessories", "Other"],
  "conditions": ["New", "Like New", "Good", "Fair"]
}
```

Error Responses:

Status Code	Condition	Error Message (JSON)
500 Internal Server Error	Unexpected server failure	{"error": "InternalServerError", "message": "Something went wrong on our end"}

Note: Browse/Search/Filter reuses the GET /api/listings endpoint documented above with query parameters, rather than duplicating it under a separate route - this avoids redundant endpoints for what is fundamentally the same resource collection with different query filters, which is consistent with REST conventions.

1.3 Core Feature 4 - Favorite / Saved Listings

Logged-in students can bookmark listings to a personal saved list, view that list, and remove items from it.

POST /api/saved/:id

Save (bookmark) a listing for the current user.

Authentication: Required - Bearer JWT **Authorization:** Any authenticated user; uniqueness is enforced per (user, listing) pair

Request Parameters / Body:

Field	Type	Required?	Notes
id	ObjectId (path param)	Yes	Id of the listing to save

Success Response: 201 Created

```
{ "saved": { "_id": "665f2...", "user": "665f0...", "listing": "665f1..." } }
```

Error Responses:

Status Code	Condition	Error Message (JSON)
401 Unauthorized	Missing/invalid JWT	{"error":"Unauthorized","message":"Invalid JWT"}
404 Not Found	Listing id does not exist	{"error":"NotFound","message":"Listing not found"}
409 Conflict	Listing already saved by this user	{"error":"Conflict","message":"Listing is already saved"}

GET /api/saved

Retrieve the current user's saved listings, with the full listing (and seller) details populated.

Authentication: Required - Bearer JWT **Authorization:** Returns only the authenticated user's own saved items (enforced via req.user._id, not a client-supplied id)

Success Response: 200 OK

```
{ "saved": [ { "_id": "665f2...", "listing": { "title": "Calculus...", "price": 45, "status": "active" } } ] }
```

Error Responses:

Status Code	Condition	Error Message (JSON)
401 Unauthorized	Missing/invalid JWT	{"error":"Unauthorized","message":"Invalid JWT"}

DELETE /api/saved/:id

Remove a listing from the current user's saved list.

Authentication: Required - Bearer JWT **Authorization:** Only removes the saved-listing row owned by the authenticated user

Request Parameters / Body:

Field	Type	Required?	Notes
id	ObjectId (path param)	Yes	Id of the listing to unsave

Success Response: 200 OK

```
{ "message": "Listing removed from saved list" }
```

Error Responses:

Status Code	Condition	Error Message (JSON)
401 Unauthorized	Missing/invalid JWT	{"error":"Unauthorized","message":"Invalid JWT"}

404 Not Found	Listing was not in the user's saved list	<code>{"error":"NotFound","message":"This listing is not in your saved list"}</code>
----------------------	--	--

2. Security-Related Endpoints

This section documents endpoints dedicated exclusively to authentication, authorization, and data validation, as required by the assignment. These are intentionally separate from the three core features above, since the assignment specifies that login and registration are not core features.

2.1 Authentication

POST /api/auth/register

Register a new student account. Restricted to university email addresses to preserve CampusCart's trust model.

Authentication: Not required (public) **Authorization:** N/A - always creates a 'student' role account; role cannot be supplied by the client (see Section 2.4 - mass assignment mitigation)

Request Parameters / Body:

Field	Type	Required?	Notes
fullName	String	Yes	1-100 characters
universityEmail	String	Yes	Valid email format; must end in a university domain (e.g. .ca/.edu)
studentId	String	No	Up to 20 characters
password	String	Yes	Minimum 8 characters; hashed with bcrypt (cost factor 12) before storage

Success Response: 201 Created

```
{
  "user": { "id": "665f0...", "fullName": "Jane Smith", "universityEmail":
"j.smith@dal.ca", "role": "student" },
  "token": "eyJhbGciOiJIUzI1NiIs..."
}
```

Error Responses:

Status Code	Condition	Error Message (JSON)
400 Bad Request	Invalid email format, non-university email, or password < 8 characters	{"error":"ValidationError","message":"One or more fields are invalid","details":[{"field":"password","message":"Password must be at least 8 characters long"}]}
409 Conflict	An account with this email already exists	{"error":"Conflict","message":"An account with this email already exists"}

429 Too Many Requests	Rate limit exceeded (20 requests / 15 min / IP)	{"error":"TooManyRequests","message":"Too many attempts, please try again later"}
------------------------------	---	---

POST /api/auth/login

Authenticate with university email + password and receive a signed JWT access token.

Authentication: Not required (public) **Authorization:** N/A

Request Parameters / Body:

Field	Type	Required?	Notes
universityEmail	String	Yes	
password	String	Yes	

Success Response: 200 OK

```
{
  "user": { "id": "665f0...", "fullName": "Jane Smith", "role": "student" },
  "token": "eyJhbGciOiJIUzI1NiIs..."
}
```

Error Responses:

Status Code	Condition	Error Message (JSON)
400 Bad Request	Missing email or password	{"error":"ValidationError","message":"One or more fields are invalid"}
401 Unauthorized	Email not found or password incorrect (identical message for both, to avoid leaking which emails are registered)	{"error":"InvalidCredentials","message":"Email or password is incorrect"}
403 Forbidden	Account has been suspended by an admin	{"error":"Forbidden","message":"This account has been suspended. Contact an administrator."}
429 Too Many Requests	Rate limit exceeded - mitigates credential-stuffing / brute force	{"error":"TooManyRequests","message":"Too many attempts, please try again later"}

GET /api/auth/me

Return the profile of the currently authenticated user, resolved from the JWT (never trusts a client-supplied user id).

Authentication: Required - Bearer JWT **Authorization:** Self only

Success Response: 200 OK

```
{ "user": { "id": "665f0...", "fullName": "Jane Smith", "role": "student", "status": "active" } }
```

Error Responses:

Status Code	Condition	Error Message (JSON)
401 Unauthorized	Missing, invalid, or expired JWT	{"error": "TokenExpired", "message": "JWT has expired, please log in again"}

POST /api/auth/forgot-password

(Planned supporting endpoint - see note below) Issues a short-lived, single-use password reset token and emails a reset link to the user's university address.

Authentication: Not required (public) **Authorization:** N/A

Request Parameters / Body:

Field	Type	Required?	Notes
universityEmail	String	Yes	

Success Response: 200 OK

```
{ "message": "If that email exists, a reset link has been sent." }
```

Error Responses:

Status Code	Condition	Error Message (JSON)
400 Bad Request	Invalid email format	{"error": "ValidationError", "message": "One or more fields are invalid"}

POST /api/auth/reset-password

(Planned supporting endpoint) Consumes a reset token and sets a new password; the token is single-use and expires after 30 minutes.

Authentication: Not required (the resetToken itself acts as a one-time credential) **Authorization:** N/A

Request Parameters / Body:

Field	Type	Required?	Notes
resetToken	String	Yes	Raw token from the emailed link; only its bcrypt hash is stored server-side
newPassword	String	Yes	Minimum 8 characters

Success Response: 200 OK

```
{ "message": "Password updated successfully" }
```

Error Responses:

Status Code	Condition	Error Message (JSON)
400 Bad Request	Weak password	{"error": "ValidationError", "message": "Password must be at least 8 characters long"}
401 Unauthorized	Token invalid, already used, or expired	{"error": "InvalidToken", "message": "This reset link is invalid or has expired"}

Note on scope: /api/auth/register, /api/auth/login, and /api/auth/me are fully implemented in the source code submitted with this assignment (see Section 4). /api/auth/forgot-password and /api/auth/reset-password are documented to the same level of rigor (including the PasswordResetToken entity in Section 3) as a planned extension, since the assignment scope for live implementation is limited to two endpoints from one core feature.

2.2 Authorization

CampusCart uses two complementary authorization mechanisms layered on top of authentication:

- **Role-Based Access Control (RBAC):** every JWT encodes the user's role (student or admin). The `requireRole(...)` middleware rejects requests from a role that is not permitted to call that route - for example, only student/admin roles may create a listing in the first place, and a hypothetical future 'banned' role would be rejected entirely at the `requireAuth` stage.
- **Resource-ownership Access Control List (ACL):** for actions on an existing resource (editing/deleting a listing, managing a saved-list entry), the API does not rely on role alone. It loads the resource, compares its `seller/user` field against `req.user._id` (taken from the verified JWT, never from the request body or URL), and only allows the action if the requester owns the resource or is an admin. This directly mitigates the OWASP 'Broken Access Control' risk of a user editing or deleting someone else's data by guessing an id.

GET /api/admin/listings

(Documented - Admin Dashboard core feature, not one of the 3 selected features, but included here as it is a security-relevant, role-gated endpoint) List all listings, including flagged/removed ones, for moderation.

Authentication: Required - Bearer JWT **Authorization:** role=admin only

Request Parameters / Body:

Field	Type	Required?	Notes
status	String (query)	No	Filter by active sold flagged removed

Success Response: 200 OK

```
{ "listings": [ { "_id": "665f1...", "status": "flagged" } ] }
```

Error Responses:

Status Code	Condition	Error Message (JSON)
-------------	-----------	----------------------

403 Forbidden	Authenticated but not an admin	{"error":"Forbidden","message":"Requires one of the following roles: admin"}
----------------------	--------------------------------	--

PATCH /api/admin/users/:id/status

(Documented) Suspend, warn, or reactivate a user account, e.g. after a policy violation.

Authentication: Required - Bearer JWT **Authorization:** role=admin only

Request Parameters / Body:

Field	Type	Required?	Notes
status	String (enum)	Yes	active warned suspended

Success Response: 200 OK

```
{ "user": { "id": "665f0...", "status": "suspended" } }
```

Error Responses:

Status Code	Condition	Error Message (JSON)
403 Forbidden	Not an admin	{"error":"Forbidden","message":"Requires one of the following roles: admin"}
404 Not Found	User id does not exist	{"error":"NotFound","message":"User not found"}

2.3 Data Validation Strategy

Every write endpoint validates input at two layers before it ever reaches the database:

- Layer 1 - express-validator at the route level: checks types, lengths, enum membership, email format, and numeric ranges, and rejects the request with 400 Bad Request and a field-level error list before the controller runs any business logic.
- Layer 2 - Mongoose schema validation at the model level: enforces required fields, enum values, and types again at the data layer, so the database can never persist an invalid document even if a future endpoint forgets to call the validator middleware.

2.4 Mitigating Security Risks Covered in Class

Risk	Mitigation in CampusCart's API
Injection (NoSQL injection)	Mongoose's query builder parameterizes all queries, so user input is never concatenated into a query string. The express-mongo-sanitize middleware additionally strips any keys starting with '\$' or containing '.' from req.body/req.query/req.params, so an attacker

	cannot send {"universityEmail": {"\$gt": ""}} to bypass a login check or filter.
Broken Access Control	Ownership checks (Section 2.2) ensure a JWT only grants access to the holder's own resources; RBAC middleware separately gates admin-only routes. Mongo ObjectIds are validated with isMongoid() so a malformed id cannot be used to probe internal behaviour.
Broken Authentication / credential stuffing	Passwords are hashed with bcrypt (cost factor 12, salted automatically) - plaintext passwords are never stored or logged. Login and register are rate-limited (20 requests / 15 minutes / IP) via express-rate-limit. JWTs expire after 1 hour, limiting the blast radius of a stolen token.
Mass assignment	Controllers whitelist exactly which fields can be set from the request body (e.g. role is hard-coded to 'student' on register and can never be supplied by the client; updateListing only copies a fixed allow-list of fields), rather than blindly spreading req.body into a Mongoose document.
Sensitive data exposure	passwordHash is excluded from every API response via .select('-passwordHash') and is never included in the JSON serialization of a User. Helmet sets secure HTTP headers (X-Content-Type-Options, no X-Powered-By, etc.).
Cross-Origin misuse	CORS is restricted to an explicit allow-list of origins (the deployed frontend URL) via the CORS_ORIGIN environment variable, rather than reflecting any origin.

3. Data Integration Strategy

CampusCart persists data in MongoDB (MongoDB Atlas in production) using Mongoose schemas. The diagram below sketches the four collections needed to support the three core features documented in Section 1 plus the security entities from Section 2.

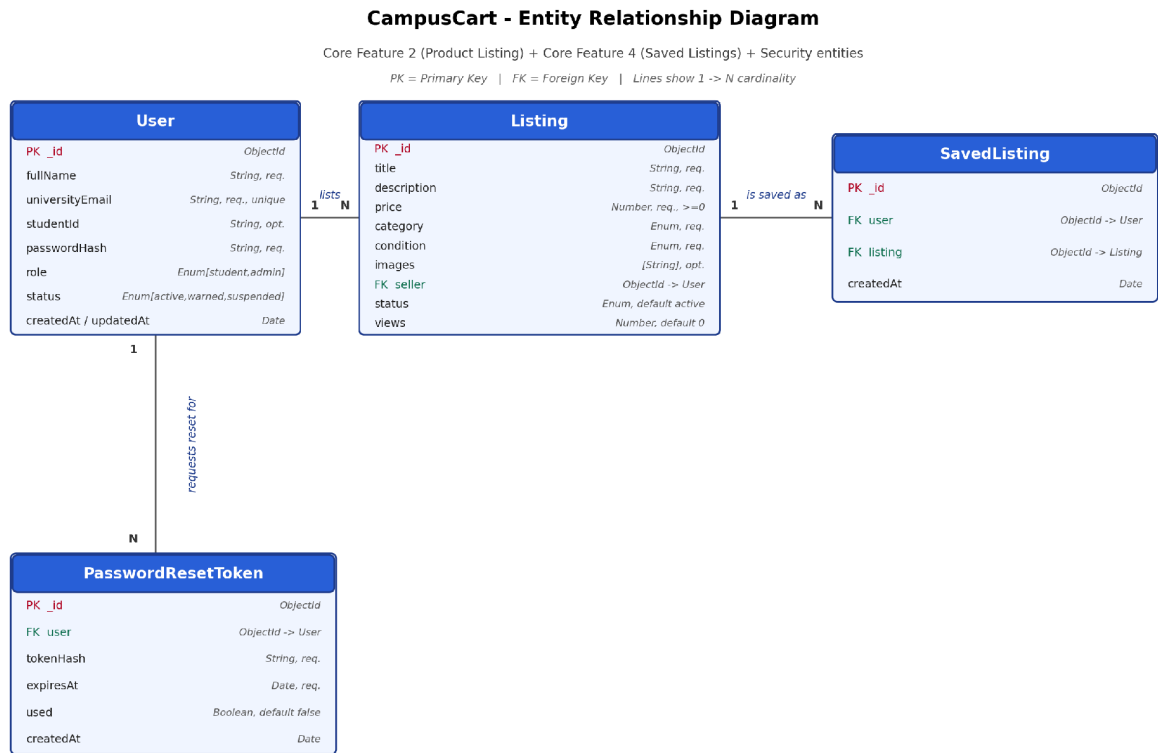


Figure 1. CampusCart Entity Relationship Diagram

3.1 Entity Definitions

User

Field	Type	Required?	Notes
_id	ObjectId (PK)	Auto	MongoDB-generated primary key
fullName	String	Required	Max 100 characters
universityEmail	String	Required, unique	Validated email format; must be a university domain
studentId	String	Optional	Self-reported student id
passwordHash	String	Required	bcrypt hash; never returned in API responses

role	Enum	Required	student (default) admin
status	Enum	Required	active (default) warned suspended
createdAt / updatedAt	Date	Auto	Mongoose timestamps

Listing

Field	Type	Required?	Notes
_id	ObjectId (PK)	Auto	
title	String	Required	Max 120 characters
description	String	Required	Max 2000 characters
price	Number	Required	>= 0
category	Enum	Required	Textbooks Electronics Furniture Accessories Other
condition	Enum	Required	New Like New Good Fair
images	Array<String>	Optional	Up to 6 URLs
seller	ObjectId (FK -> User)	Required	Owner of the listing
status	Enum	Required	active (default) sold flagged removed
views	Number	Auto	Default 0; incremented on GET /api/listings/:id
createdAt / updatedAt	Date	Auto	

SavedListing

Field	Type	Required?	Notes
_id	ObjectId (PK)	Auto	
user	ObjectId (FK -> User)	Required	Who saved the listing
listing	ObjectId (FK -> Listing)	Required	What was saved
createdAt	Date	Auto	Compound unique index on (user, listing) prevents duplicate saves

PasswordResetToken

Field	Type	Required?	Notes
_id	ObjectId (PK)	Auto	
user	ObjectId (FK -> User)	Required	

tokenHash	String	Required	bcrypt hash of the emailed token; raw token is never stored
expiresAt	Date	Required	30 minutes after creation
used	Boolean	Required	Default false; set true after a successful reset (single-use)
createdAt	Date	Auto	

3.2 CRUD Operations Mapping

The table below maps each entity to the CRUD (and related) operations performed against it, and the exact API endpoint the client calls to trigger that operation.

Entity	Operation	Endpoint	Who can perform it
User	Create	POST /api/auth/register	Public
User	Read (self)	GET /api/auth/me	Owner
User	Update (status)	PATCH /api/admin/users/:id/status	Admin
Listing	Create	POST /api/listings	Authenticated student/admin
Listing	Read (one)	GET /api/listings/:id	Public
Listing	Read (many / search / filter / sort / paginate)	GET /api/listings	Public
Listing	Update	PUT /api/listings/:id	Owner or admin
Listing	Delete	DELETE /api/listings/:id	Owner or admin
Listing	Moderate (read all incl. flagged)	GET /api/admin/listings	Admin
SavedListing	Create	POST /api/saved/:id	Authenticated user
SavedListing	Read (own list)	GET /api/saved	Owner
SavedListing	Delete	DELETE /api/saved/:id	Owner
PasswordResetToken	Create	POST /api/auth/forgot-password	Public
PasswordResetToken	Consume / Update (used=true)	POST /api/auth/reset-password	Holder of raw token

4. Implementation and Deployment

Two endpoints from Core Feature 2 (Product Listing Management) were implemented end-to-end and deployed live, with the supporting registration/login endpoints needed to obtain a JWT and complete the workflow:

- POST /api/listings - create a new product listing (requires authentication)
- GET /api/listings - browse/search/filter/sort/paginate listings (public)
- Supporting endpoints enabling the full workflow: POST /api/auth/register, POST /api/auth/login

4.1 Technology Stack

- Runtime/Framework: Node.js + Express.js
- Database: MongoDB Community Edition 8.0, accessed via Mongoose
- Authentication: JSON Web Tokens (jsonwebtoken), bcryptjs for password hashing
- Validation/Security middleware: express-validator, express-mongo-sanitize, helmet, cors, express-rate-limit
- Process manager: PM2 (keeps the API running continuously and restarts it automatically on crash or instance reboot)
- Hosting (backend + database): a single AWS EC2 t3.micro instance (Ubuntu 24.04 LTS)

Note on hosting choice: the database was initially provisioned on MongoDB Atlas (the architecture described in the Group 8 proposal) and was fully functional from a local development machine. However, the TLS handshake between Atlas and every containerized PaaS host tested (Render and Railway) consistently failed with a low-level SSL alert specific to MongoDB's native wire protocol on port 27017, while every other outbound HTTPS connection from those same hosts (GitHub, npm, Atlas's own web dashboard) succeeded normally. After ruling out credentials, IP allow-listing, cluster health, Node.js version, and TLS protocol version as causes, the API and database were deployed together on a single AWS EC2 instance so that the connection is local (localhost) rather than crossing the public internet to Atlas - eliminating the failing network path entirely while keeping the same MongoDB/Mongoose codebase unchanged.

4.2 Live Endpoint URLs

Endpoint	Method	Live URL
Health check	GET	http://3.15.207.244:5000/api/health
Register	POST	http://3.15.207.244:5000/api/auth/register
Login	POST	http://3.15.207.244:5000/api/auth/login
Create Listing [REQUIRED]	POST	http://3.15.207.244:5000/api/listings
Browse Listings [REQUIRED]	GET	http://3.15.207.244:5000/api/listings

The server is kept running continuously via PM2 with auto-restart on crash and on instance reboot, so these URLs should remain live for grading.

4.3 Source Code

GitHub repository: <https://github.com/maheks1713-bit/Assignment---2>

The backend source code is provided in the /backend directory of this submission, containing the Express app (src/app.js, src/server.js), Mongoose models (src/models/), JWT/RBAC middleware (src/middleware/auth.js), input-validation middleware (src/middleware/validate.js), route/controller pairs for auth, listings, and saved listings, a seed script (src/seed/seed.js), and an automated smoke test that exercises every endpoint against an in-memory MongoDB instance (src/seed/memoryServerSmokeTest.js, runnable via `npm run test:memory`).

4.4 Database Source Files

campuscart-sample-data.json (included with this submission) contains a full export of the seeded User, Listing, and SavedListing collections, taken directly from the live deployed database on the EC2 instance by running `npm run seed` there. This satisfies the requirement to provide the source files for the database with data in it.

4.5 Postman Evidence

A ready-to-import Postman collection (backend/postman/CampusCart.postman_collection.json) is included, pre-configured with baseUrl set to the live URL above. Import it and run the requests in order (01 through 11) to reproduce every screenshot listed below.

Required screenshots (annotate each with the step name before pasting):

GET

{{baseUrl}} /api/listings/000000000000000000000000

Send

Docs

Params

Authorization

Headers (6)

Body

Scripts

Settings

Cookies

Query Params

	Key	Value	Description	...	Bulk Edit
	Key	Value	Description		

Body

Cookies

Headers (19)

Test Results

404 Not Found

122 ms

964 B

Save Response

JSON

Preview

Debug with AI

```
1 {
2   "error": "NotFound",
3   "message": "Listing not found"
4 }
```

GET

{{baseUrl}} /api/listings/ {{listingId}}

Send

Docs

Params

Authorization

Headers (6)

Body

Scripts

Settings

Cookies

Query Params

	Key	Value	Description	...	Bulk Edit
	Key	Value	Description		

Body

Cookies

Headers (19)

Test Results

200 OK

132 ms

1.33 KB

Save Response

JSON

Preview

Visualize

```
1 {
2   "listing": {
3     "_id": "6a42449c6ffb41ca4ac31a7d",
4     "title": "Calculus: Early Transcendentals 8th Ed.",
5     "description": "Gently used copy, spine intact, minimal highlighting.",
6     "price": 45,
7     "category": "Textbooks",
8     "condition": "Like New",
9     "images": [],
10    "seller": {
11      "_id": "6a423ef4c100ab92624f22e3",
12      "fullName": "Jane Smith",
13      "universityEmail": "j.smith@dal.ca"
14    },
15    "status": "active",
16    "views": 1,
```

GET

{{baseUrl}}/api/listings?category=Textbooks&sort=price_asc&page=1&limit=10

Send

Docs

Params

Authorization

Headers (6)

Body

Scripts

Settings

Cookies

☒	sort	price_asc	
☒	page	1	
☒	limit	10	
	Key	Value	Description

Body

Cookies

Headers (19)

Test Results

200 OK

132 ms

1.87 KB

Save Response

JSON

Preview

Visualize

```
1 {
2   "listings": [
3     {
4       "_id": "6a423ef4c100ab92624f22e9",
5       "title": "Calculus: Early Transcendentals 8th Ed.",
6       "description": "Gently used copy, spine intact, no missing pages, minimal highlighting.
7         Perfect for MATH 1000/2001.",
8       "price": 45,
9       "category": "Textbooks",
10      "condition": "Like New",
11      "images": [],
12      "seller": {
13        "_id": "6a423ef4c100ab92624f22e3",
14        "fullName": "Jane Smith",
15        "universityEmail": "j.smith@dal.ca"
16      }
17    }
18  ]
19 }
```

POST

{{baseUrl}}/api/listings

Send

Docs

Params

Authorization

Headers (10)

Body

Scripts

Settings

Cookies

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body

Cookies

Headers (19)

Test Results

400 Bad Request

124 ms

1.19 KB

Save Response

JSON

Preview

Debug with AI

```
1 {
2   "error": "ValidationError",
3   "message": "One or more fields are invalid",
4   "details": [
5     {
6       "field": "title",
7       "message": "title is required"
8     },
9     {
10      "field": "price",
11      "message": "price must be a number >= 0"
12    },
13    {
14      "field": "category",
15      "message": "category must be one of: Textbooks, Electronics, Furniture, Accessories, Other"
16    }
17  ]
18 }
```

POST

{{baseUrl}}/api/listings

Send

Docs

Params

Authorization

Headers (9)

Body

Scripts

Settings

Cookies

Query Params

	Key	Value	Description	Bulk Edit
	Key	Value	Description	

Body

Cookies

Headers (19)

Test Results

401 Unauthorized

129 ms

1022 B

Save Response

JSON

Preview

Pass the correct auth credentials

```
1 {
2   "error": "Unauthorized",
3   "message": "Missing or malformed Authorization header. Expected: Bearer <token>"
4 }
```

POST

{{baseUrl}}/api/auth/login

Send

Docs

Params

Authorization

Headers (9)

Body

Scripts

Settings

Cookies

Query Params

	Key	Value	Description	Bulk Edit
	Key	Value	Description	

Body

Cookies

Headers (23)

Test Results

200 OK

536 ms

1.35 KB

Save Response

JSON

Preview

Visualize

```
1 {
2   "user": {
3     "id": "6a423ef4c100ab92624f22e3",
4     "fullName": "Jane Smith",
5     "universityEmail": "j.smith@dal.ca",
6     "role": "student",
7     "status": "active",
8     "createdAt": "2026-06-29T09:46:28.028Z"
9   },
10  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiI2YTQyM2VmNGMxMDBhYjkyNjI0ZjIyZTMiLCJyb2x1Ijoic3R1ZGVudCIsIm1hdCI6MTc4MjcyNzgyMSwiZXhwIjoxNzgyNzgyNzgyNDI0fQ.NM4nhtQp3K37U6_BBuB0vx0a35z3h4gK-uyLANeDEf4"
11 }
```

POST

{{baseUrl}}/api/auth/login

Send

Docs

Params

Authorization

Headers (9)

Body

Scripts

Settings

Cookies

Query Params

	Key	Value	Description	...	Bulk Edit
	Key	Value	Description		

Body

Cookies

Headers (23)

Test Results

401 Unauthorized

488 ms

1.06 KB

Save Response

JSON

Preview

Pass the correct auth credentials

```
1 {
2   "error": "InvalidCredentials",
3   "message": "Email or password is incorrect"
4 }
```

POST

{{baseUrl}}/api/auth/register

Send

Docs

Params

Authorization

Headers (9)

Body

Scripts

Settings

Cookies

Query Params

	Key	Value	Description	...	Bulk Edit
	Key	Value	Description		

Body

Cookies

Headers (23)

Test Results

400 Bad Request

64 ms

1.3 KB

Save Response

JSON

Preview

Debug with AI

```
1 {
2   "error": "ValidationError",
3   "message": "One or more fields are invalid",
4   "details": [
5     {
6       "field": "universityEmail",
7       "message": "A valid university email is required"
8     },
9     {
10      "field": "universityEmail",
11      "message": "Please use your university email address"
12    },
13    {
14      "field": "password",
15      "message": "Password must be at least 8 characters long"
16    }
17  ]
18 }
```

POST

{{baseUrl}}/api/auth/register

Send

Docs

Params

Authorization

Headers (9)

Body

Scripts

Settings

Cookies

Query Params

	Key	Value	Description	Bulk Edit
	Key	Value	Description	

Body

Cookies

Headers (23)

Test Results

409 Conflict

132 ms

1.06 KB

Save Response

JSON

Preview

Debug with AI

```
1 {
2   "error": "Conflict",
3   "message": "An account with this email already exists"
4 }
```

POST

{{baseUrl}}/api/auth/register

Send

Docs

Params

Authorization

Headers (9)

Body

Scripts

Settings

Cookies

Query Params

	Key	Value	Description	Bulk Edit
	Key	Value	Description	

Body

Cookies

Headers (23)

Test Results

409 Conflict

118 ms

1.06 KB

Save Response

JSON

Preview

Debug with AI

```
1 {
2   "error": "Conflict",
3   "message": "An account with this email already exists"
4 }
```

5. References

- OWASP Foundation. (2021). OWASP Top 10:2021. <https://owasp.org/Top10/>
- Fielding, R. T. (2000). Architectural Styles and the Design of Network-based Software Architectures (REST). University of California, Irvine.
- MongoDB, Inc. (2024). Mongoose Documentation. <https://mongoosejs.com/docs/>
- Express.js. (2024). Express API Reference. <https://expressjs.com/en/4x/api.html>
- jwt.io. (2024). Introduction to JSON Web Tokens. <https://jwt.io/introduction>
- Group 8. (2026). CampusCart Project Proposal. Dalhousie University, CSCI 5709.